

Transformers.js と LiteRT をつかって Gemma をブラウザで動かそう

Gemma Meetup 2026 (GDG Tokyo)

太田 満久 (おおたまん)

[@ohtaman](#) / Ubie Lab 所長, GDE (AI / Cloud AI)

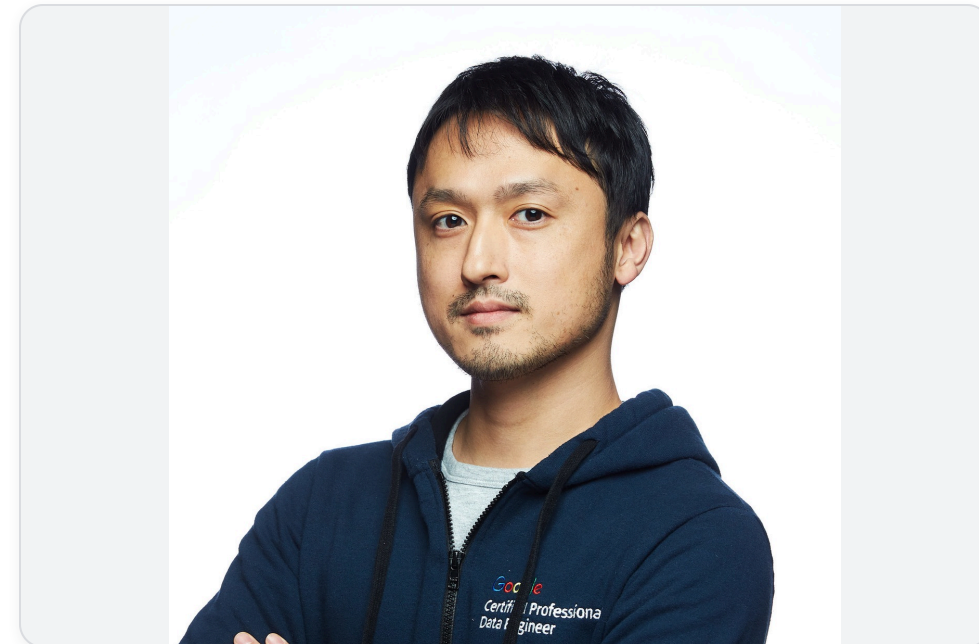


スライド資料

1. 自己紹介

おおたまん

- 所属:
 - Ubie. Inc. (Ubie Lab 所長)
 - Google Developer Expert (AI / Cloud AI)
- 活動:
 - 数理最適化コミュニティ Casual Optimization
 - 書籍執筆
- X: [@ohtaman](#)
- GitHub: github.com/ohtaman



2-1. Google I/O Connect China 2026

中国・APAC のデベロッパーに向けた Google I/O の地域旗艦イベント

- **開催:** 2026.8.12-13 上海世博中心
- Google I/O の中国版イベント
- 現地・APACのデベロッパー中心に
2,000 名以上が参加



イベント公式サイト



I/O Connect China 2026 メイン会場 集合写真

2-2. 「エッジ推論」と「オープンモデル」の重視

AndroidとWebで6割。エッジ推論やオープンモデルのセッションが多い

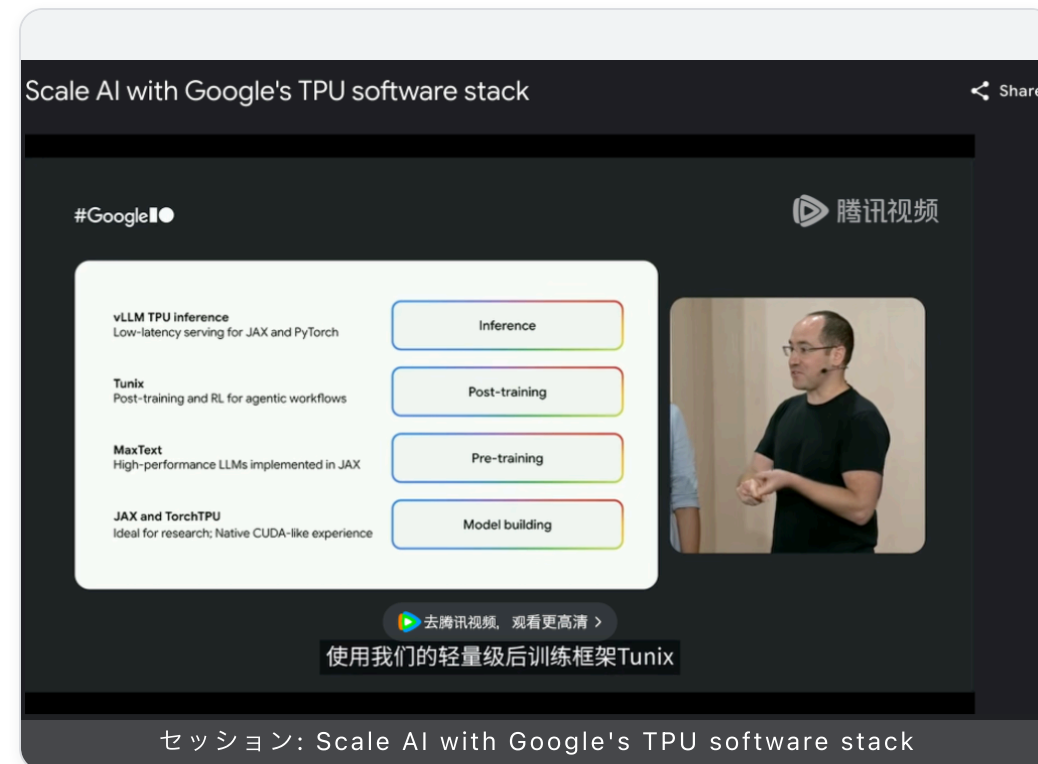
比較項目	Google I/O (Mountain View / 72枠)	Google I/O Connect China (86枠)
カテゴリ構成	- Android: 43% (31枠) - Cloud: 21% (15枠) - AI: 19% (14枠) - Web: 17% (12枠)	- Android: 40% (34枠) - Web: 21% (18枠) - AI: 21% (18枠) - Cloud: 19% (16枠) 👉 Android + Web で約 60%
主な AI 関連トピック	- Gemini API / Google AI Studio - Vertex AI / Workspace MCP - Android Studio / DevTools AI	- Gemma (オープンモデル) - LiteRT / LiteRT-LM (エッジ推論) - TPU ソフトウェア (MaxText / vLLM)
推論・適用の方向性	クラウド API 連携、開発ツールへの AI 統合	端末内 (On-device) 推論、オープンモデル活用

※ 共通の主要技術分野（Android / Cloud / AI / Web）のセッションを対象に集計（対談・基調等を除く）

2-3. Scale AI with Google's TPU software stack

モデル構築から学習・推論まで、ソフトウェアスタックが体系化されている

- セッション概要:
 - Google TPU を支えるオープンソース群の全体像を解説
- モデルライフサイクルを支える 4 つの層:
 - **Inference: vLLM TPU** (低遅延推論)
 - **Post-training: Tunix** (事後学習・LoRA・RL)
 - **Pre-training: MaxText** (事前学習)
 - **Model building: JAX / TorchTPU**



2-4. 展示ブース: 多様なデバイスで動く Gemma のデモ

モバイル、モバイル、ARメガネなど、多様なデバイスで Gemma のデモが展示されていた

- 視覚障害者向け画面読み上げ「VisAware」
 - スクリーンリーダー用のAI視覚認識プラグイン
 - 画面内の画像・図表をGemmaで解説
- in BrowserでのGemmaの利用例
 - Transformers.js や LiteRT-LM Web API
- その他多様なデバイスでの活用
 - ブラウザやスマホ、各種エッジデバイスなど、様々なデバイスで Gemma を実行



3. ブラウザで動かす理由

手軽な配布、ブラウザ機能の利用、推論コストのユーザー端末への転嫁

- ① 配布の容易さ
 - Python や環境構築が不要。URL を開くだけで即利用可能。
- ② ブラウザ機能との連携
 - カメラ、マイク、画面共有などブラウザの機能をシームレスに利用
- ③ 計算コストの転嫁
 - サーバー側の推論コストをユーザーへ転嫁

4. ブラウザ LLM の実行アプローチ

組み込み API もしくは JSライブラリを利用する

- **ブラウザ組み込み (Gemini Nano / Chrome Prompt API)**
 - Chrome の標準機能のため、実装が容易
 - 利用モデルは固定 (Gemini Nano) でチューニング不可
- **JSライブラリ (Transformers.js / LiteRT-LM / WebLLM)**
 - 外部ライブラリやモデル管理が必要
 - モデルを自由に選択できる (Gemma 等)
 - 自前チューニングモデルを利用することも可能

5. Transformers.js の思想と実装

Hugging Face の巨大資産をそのまま Web へ

- HF Hub 上の多様な ONNX モデルを直接ブラウザで利用可能
- WebGPU (ONNX Runtime Web) によるハードウェアアクセラレーション

呼び出しコード（実例）：

```
import { pipeline } from '@huggingface/transformers';

const pipe = await pipeline('text-generation', 'onnx-community/gemma-4-E2B-it-ONNX', {
  device: 'webgpu', dtype: 'q4f16'
});

const result = await pipe([
  { role: 'user', content: 'ブラウザで LLM を動かすメリットを 3 つ教えて' },
], { max_new_tokens: 1024 });
```

6. LiteRT-LM の思想と実装

TFLite の後継。Android / iOS / Web を単一モデルで

- Google のオンデバイス推論基盤（旧 TensorFlow Lite）の後継
- 高度な最適化により、実用的なパフォーマンス
- 単一の `.litemlm` ファイルが、モバイルとブラウザでそのまま共通動作

呼び出しコード（実例）：

```
import { Engine } from '@litemlm/core';

const engine = await Engine.create({ model: 'gemma-4-E2B-it-web.litemlm' });
const chat = await engine.createConversation();
const stream = chat.sendMessageStreaming('ブラウザで LLM を動かすメリットを 3 つ教えて');
for await (const chunk of stream) {
  updateUI(chunk.content?.[0]?.text); // リアルタイムストリーミング
}
```

7. 量子化とアーキテクチャの内部比較

Web エコシステム重視の Transformers.js と、単一バイナリ・最適化重視の LiteRT-LM

比較軸	Transformers.js	LiteRT-LM
ベースモデル	Gemma 4 E2B-it	Gemma 4 E2B-it
量子化方式	INT4 (q4f16) (Optimum / MatMulNBits)	4-bit 級 混合量子化 (2/4/8-bit Mixed + mmap)
パッケージ構成	ONNX + 外部データ + トークナイザ (複数ファイル)	重み・トークナイザ・メタデータが 単一バイナリ
トークナイザ	JavaScript / Wasm 分離実装 (tokenizers.js)	C++ ネイティブ SentencePiece 内蔵
推論エンジン	ONNX Runtime Web (WebGPU / WGSL)	LiteRT C++ コア (WebAssembly + WebGPU)
アーキテクチャ設計	Web / オープンエコシステム (疎結合) HF の膨大な資産を柔軟にパイプライン化	モバイル / 組み込み直系 (密結合) 配布事故ゼロの単一バイナリ & 極限最適化

👉 HF 資産の柔軟なパイプライン化 vs 配布事故ゼロの単一バイナリ極限最適化

8. 【デモ】 Transformers.js と LiteRT-LM の動作比較

同一モデル（Gemma 4 E2B 4-bit）のブラウザ上での動作比較



👉 デモ公開中

ohtaman.github.io/ai-in-browser-demo/06_comparison_arena/index.html

Transformers.js vs LiteRT-LM 比較 Playground
同等スペック（Gemma 4 E2B / 4-bit 量子化）のブラウザ WebGPU 直接対決・公平比較

共通プロンプト入力:
JavaScript でフィボナッチ数を計算する関数を書いて

プリセット: ブラウザ推論のメリット Gemma 4 の特徴 JSコード生成 Max Tokens: 2048

完了! Transformers: 39.1 tok/s | LiteRT: 55.3 tok/s [順次ベンチマーク実行 \(Sequential\)](#)

* 公平な測定のため、GPU/ソースを混同しない「順次 (A → B) 実行」を行います。

Transformers.js Hugging Face ロード済み		LiteRT-LM Google AI Edge ロード済み	
ベースモデル	Gemma 4 E2B-It	ベースモデル	Gemma 4 E2B-It
量子化仕様	INT4 (q4f16) / MatMulNbits	量子化仕様	4-bit 量 混合量子化 (2/4/8-bit Mixed)
推論エンジン	ONNX Runtime Web (WebGPU)	推論エンジン	LiteRT C++ コア (Wasm + WebGPU)
パッケージ	ONNX + 分割外部データ (約1.5GB)	パッケージ	単一バイナリ (LiteRT 約1.9GB)
生成速度	39.1 tok/s	生成速度	55.3 tok/s
初回トークン (TTF)	369 ms	初回トークン (TTF)	180 ms
生成トークン数	1155	生成トークン数	1342
総所要時間	29.52 s	総所要時間	24.26 s

生成出力: [単独実行](#)

はい、JavaScriptでフィボナッチ数を計算する関数をいくつか紹介します。

最も一般的で効率的な**再帰的な方法**、**イテレーション（ループ）を使った方法**、そして**メモ化（動的計画法）を使った方法**の3つの例を示します。

1. 再帰的な方法 (最も直感的)

フィボナッチ数列の定義 ($F(n) = F(n-1) + F(n-2)$) をそのまま実装

はい、JavaScriptでフィボナッチ数を計算する関数をいくつか紹介します。

最も一般的で効率的なのは、**再帰**を使う方法と、**イテレーション（ループ）**を使う方法です。

1. 再帰関数による方法 (直感的)

フィボナッチ数列の定義 ($F(n) = F(n-1) + F(n-2)$) を直接コードに反映させる方法です。

実測結果: フィボナッチ関数生成 (1,155 vs 1,342 tok)

9. 特徴と実測値の比較

LiteRT-LM は速度・TTFTで優勢。ただし、対応モデルの数や柔軟性では Transformers.js に分がある

比較ポイント	Transformers.js	LiteRT-LM
生成速度（実測）	39.1 tok/s（十分実用的）	55.3 tok/s（約 1.4 倍 高速 / +41%）
初回応答（TTFT）	369 ms	180 ms（約 2 倍 高速 / -51% 短縮）
配布パッケージ	ONNX + データ + トークナイザ（複数ファイル）	単一バイナリ（ <code>.litertlm</code> 1本で完結）
対応モデル	豊富（HF 上の多様なオープンモデルをコミュニティが変換）	限定的（Google 公式対応の Gemma 等が中心）
機能の成熟度	成熟（マルチモーダル対応、Structured Output 等）	発展途上（Web 版は Structured Output 未対応など）
どんな時に選ぶ？	多様なモデルや構造化出力を扱いたい時 音声・画像など他タスクと連携したい時	Gemma で最速の推論速度・低遅延を出したい時 Android / iOS アプリとモデルを共通化したい時

👉 純粋な推論性能と単一バイナリ配布は LiteRT-LM が優勢。対応モデルの広さや Web での機能成熟度（Structured Output 等）は Transformers.js に分がある。

10. ブラウザ推論における実用上の考慮点

初回ダウンロードやメモリの圧迫、計算負荷への配慮が必要

- ① 初回ダウンロードのオーバーヘッド（数GB）
 - 2回目以降はキャッシュ可能だが初回の待機時間が体験を左右
- ② モバイル端末でのメモリ上限（タブの強制終了）
 - スマホのブラウザは1タブあたりのメモリ制限が厳しい
- ③ 計算負荷と実行環境への配慮
 - ローカル推論による**端末の発熱・バッテリー消費**
 - メインスレッドをブロックしないための **Web Worker の活用**が必須

11. まとめ

- ① I/O Connect China から見たエッジ推論の潮流
 - オンデバイス推論やオープンモデル（Gemma）への強い関心
- ② ブラウザ推論ならではの価値
 - 手軽な配布とブラウザ標準機能との連携
 - 推論コストをユーザーに転嫁
- ③ Transformers.js と LiteRT-LM の使い分け
 - **LiteRT-LM**: 単一バイナリ配布と高い推論性能
 - **Transformers.js**: 多様なモデルの選択肢と機能成熟度

ありがとうございました！

- デモ: ohtaman.github.io/ai-in-browser-demo/06_comparison_arena/index.html
- デモコード: github.com/ohtaman/ai-in-browser-demo
- スライド資料: ohtaman.github.io/slides/gemma_meetup_2026/slides.pdf